

meArm 机械臂固件程序说明

(单文件版 single/meArm/meArm.ino)

阅读说明

本文说明当前提交的单文件版固件如何完成考试的任务一~五，并单独解释四个容易看不明白的机制：示教点记录、轨迹生成、速度控制、异常处理。文中出现的行号都是 single/meArm/meArm.ino 的行号（该文件由多文件工程用脚本合并生成，功能与多文件版完全一致），读者不需要打开源代码即可理解整体逻辑；需要核对细节时，按表 1 的行号直接跳到对应模块。

硬件：Arduino Uno + 四个舵机（D9 基座 b、D7 上臂 r、D8 下臂 c、D6 夹具 f）+ 两只双轴摇杆（A0~A3）+ D13 指示灯；串口 115200-8N1。上电后机械臂先摆到待机位，随后摇杆、按键与串口三种入口都可以操作它。

表 1 单文件版 single/meArm/meArm.ino（7344 行）的模块与行号

模块（原文件名）	行号	职责
weArm_config.h	4 - 248	编译期功能开关（取放/按键/绘图）、引脚、录制与绘图的全部可调参数
constant_and_positions.h/.cpp	250 - 485 / 1792 - 2652	全局位姿 Pos（唯一被控量）、关节硬限位 servoLimit、正/反运动学、速度档位
protocol_constants.h/.cpp	487 - 577 / 2654 - 2715	串口命令字符与结果码常量表
servo_drive.h/.cpp	579 - 653 / 1662 - 1790	Timer1 顺序脉冲的 4 路舵机驱动（脉宽映射 + 20ms 帧）
move.h / move.cpp	655 - 793 / 2717 - 2932	运动基元：关节步进 moveJointStep()、坐标移动 moveToPoint()、关节速率限制
path_core.h	1557 - 1660	纯几何：折线/曲线的等分点与路径采样（与硬件无关，PC 端可测）
joystick_control.h/.cpp	1421 - 1555 / 2934 - 3456	两摇杆读取、中位自标定、按调速档位驱动关节、D13 指示灯
pick_place.h/.cpp	915 - 997 / 3458 - 3962	A/B/C 三个物体的自动取放序列（状态机 + 启动前路径校验）
button_control.h/.cpp	999 - 1132 / 5364 - 6140	四个按键（含录制/播放/回中）与它们的串口孪生命令
draw_control.h/.cpp	1134 - 1419 / 3964 - 5362	绘图：任务选择、内置图形、五点示教、轨迹生成、梯形速度、卡死保护
serial_protocol.h/.cpp	795 - 913 / 6142 - 7104	串口命令解析与一行短回复、忙守卫、单读者约束
meArm.ino	7106 - 7344	setup()/loop()、引脚绑定、舵机镜像输出、指示灯

一、总体结构与运行框架

固件的中心只有一个数据结构：Pos（constant_and_positions.h）。它同时保存四个关节角 Pos.ser.angle1..angle4 和由正运动学生成的末端坐标 Pos.rec.x/y/z。任何控制入口最后都只改关节角，再刷新一次正解，因此“角度”与“机械臂实际位置”永远严格对应，不存在两套状态互相不一致的可能。

setup() 的顺序是：1 posInit() 摆到待机位；2 serialProtocolBegin() 打开串口并打印命令表；3 joystickSetup() 标定摇杆中位；4 buttonSetup() 初始化按键；5 drawSetup() 初始化绘图模块；6 servoDriveBegin() 启动定时器并把四个通道绑到 D9/D7/D8/D6；最后 writeServo() 输出一角度。

loop() 的顺序是: serialProtocolLoop() (串口最优先, 保证命令响应及时) → pickPlaceLoop() → buttonLoop() → drawLoop() (放在按键之后是为了接收按键转交、放在摇杆之前是为了整段让位) → joystickLoop() → writeServo()。六个函数都是**非阻塞**的: 谁都没有 delay(), 一次循环只推进一步。

writeServo() 是唯一的舵机写出口。它对基座与夹具做窗口内镜像 (servoDriveWrite(0, (servoLimit.minB+servoLimit.maxB) - Pos.ser.angle1), 夹具同理), 用来把“逻辑角度”映射到实机装配方向; 逻辑角度始终是唯一真值, 改装配只动这一处。

二、任务一: 舵机控制与角度指令

1) 舵机 PWM 驱动 (servo_drive.h/.cpp, 第 579 - 653 / 1662 - 1790 行)

四个舵机由 Timer1 顺序脉冲驱动, 一帧 20ms: 中断里依次拉高/拉低四个通道的引脚, 四个脉冲连发完等帧尾, 下一帧把计数器清零并同时拉起第 0 路, 所以帧长严格是 40000 个 tick (0.5μs/tick) = 20ms。脉宽映射与 Arduino Servo 库逐位一致: 角度先夹到 0~180, 再按 $\text{angle} \times 1856 / 180 + 544$ 得到微秒, 夹到 [544, 2400] 后减去 2μs 的修整量, 最后 $(\mu s - 2) \times 2$ 就是定时器比较值。

不用现成 Servo 库的原因写在模块头: 本工程只需要“固定 4 路、固定引脚、只写角度”, 而 Servo 库为此要付出约 986 字节 flash (中断向量 376、write 248、attach 240、其余约 120), 自研实现约 300 字节, **净节约 650 字节**。芯片 flash 只有 32KB, 这一项直接决定了后三个功能能否一起编进来。

为了可测, 脉冲状态机 servoDriveStep(nowTicks, &next) 写成纯函数 (给定当前计数决定本次动作与下一次比较值), PC 端自检直接按 tick 步进它, 验证“引脚顺序、脉宽、帧长”, 时序逻辑不必上硬件才能验证。一个必须注意的坑: 帧尾比较值要在“四路都发完”那次中断里算好; 若等帧尾中断再算, 入口读到的计数已越过 40000, 16 位下溢会把下一帧推到约 52.8ms, 帧率掉到 19Hz 并抖动。

2) 软归零与关节限位 (constant_and_positions.h/.cpp)

servoLimit 是四个关节的硬限位: 基座/上臂/下臂 0~180°, 夹具 60~150°。clampServoAngles() 把越界值夹回区间 (策略由 servoLimitMode 选择“夹取”或“拒绝”, 默认夹取), 任何入口写入关节角后都会调用它。posInit() / posGetHomeAngles() 给出待机位 POS_HOME=(90, 90, 90), 对应末端 (20, 0, 20)——上电、按键 4 (串口 M 或 0) 回中、以及取放结束都走这条路径, 靠关节空间插值平滑到位, 行程中每步都用 servoSelfCheck() 检查是否越界。

3) 串口角度指令 (serial_protocol.h/.cpp, 第 795 - 913 / 6142 - 7104 行)

命令格式为 x 角度,y 角度,z 角度, 例 x10,y30,z20。这三个字母就是三个舵机的角度, 不做逆解: x 写基座 angle1、y 写上臂 angle2、z 写下臂 angle3, 夹具 angle4 保持不变。解析流程是 protoParseAxisLine() 得到三个角度和“哪些轴出现在这行里”的掩码, 再由 protoApplyAngles() 一次性写入, 然后 clampServoAngles() 夹取、recFromServo() 刷新末端坐标。只出现的轴被写入, 因此“一条指令里同时给出的几个角度”是在同一次正解刷新中同步到位的。

三条设计取舍: 允许只写一部分 (y30 只动上臂); **超出行程只夹取、不拒绝**, 例 x200 最终写 180, 这样上位机不需要先读状态就能下达任意值; 语法错误 (字母写坏、缺逗号) 回 ERR, 而忙的时候回 BUSY。每条命令都会回一行短回复, 含义见任务五与异常处理两节。

三、任务二: 摇杆手动控制 (joystick_control.h/.cpp)

两只摇杆占 A0~A3, 四路同时可用, 不需要切换模式: 左手柄左右 (A0) 转基座 b, 左手柄前后 (A1) 抬上臂 r, 右手柄前后 (A3) 俯仰下臂 c, 右手柄左右 (A2) 开合夹具 f。

中位在 joystickSetup() 里自标定: 上电瞬间连采 32 次求平均, 扣掉每个轴的静态偏差; 若某一路偏差大得离谱 (例如开机就被压住, 或超过 ±80 计数), 该路退回标准中位 512, 避免机械臂在没人碰摇杆时自己慢慢走。运行期还有一层“中位跟踪”: 当某一路的偏差落在死区以内时, 每 20

ms 把它的静态偏差朝当前读数挪一个计数，这样电位器温漂在开机几十秒后会被自动吸收，不必重新上电。

死区 JOY_DEADZONE 取 40 个 ADC 计数（约 $\pm 8\%$ 行程），把噪声峰峰（ $\pm 5 \sim \pm 15$ 计数）和温漂一次关在门外；死区以外每格关节角 = `speed.stepSize` $\times$ 偏转比例（下限 0.1° ，所以轻推就是小步），而步进间隔是固定的 `speed.stepDelayMs`（慢/中/快 = 80/40/20 ms），不再随偏转变化——推得越狠只是“一步走得更远”，节奏恒定。为避免死区边缘的残余偏差被当成推杆，起控用施密特迟滞：静止时要超过 JOY_DEADZONE + JOY_HYST = 65 计数才起控，起控后只要还大于 40 计数就继续走，手感是“推得越狠步长越大、轻推慢慢挪”。

每次步进都调用 `moveJointStep()`：它会先检查目标是否越界，越界就返回 MOVE_AT_LIMIT 并原地停住（不会顶死舵机）；正常推进后刷新正解。D13 指示灯在任一关节运动时快闪，方便一眼看出机械臂是否在动。

为什么用手柄控角度而不是控末端坐标：末端坐标可解的角度组合常超出舵机行程，摇杆推到头会突然停住；姿态也难以预判；而且夹具角本来就不参与逆解。控角度后四个关节都能被直接、连续地驱动。

四、任务三：按键控制（button_control.h/.cpp）

四个按键接 D2~D5，全部 INPUT_PULLUP（按下读到 LOW），软件消抖 25ms 且只认按下沿，一次按下只触发一次。按键 1 循环执行取放 A→B→C→A；按键 2 是录制开关；按键 3 播放；按键 4 回中。四个动作都有串口孪生命令：N R P M (0 等价于 M)，效果与按物理键完全一样。

录制：每 200ms 采样一次四个关节角与末端坐标，最多 192 条（768 字节 SRAM，约 38.4 秒）。结束录制时只看“动没动”——末端在 x/y/z 上至少有一轴走出 10 个单位才保存，否则丢弃并回 DISCARD；早期版本要求“必须大于 10 秒”，会造成正常短动作被无理由丢弃，已删除。播放时把每条 200ms 的记录再拆成 4 个 50ms 子步插值，动作比录制时更平滑；播放前若机械臂不在轨迹起点，会先平滑移到起点。

按键 1 背后的自动取放序列（pick_place.h/.cpp，第 915 - 997 / 3458 - 3962 行）

按键 1（串口 N 或 A/B/C 直接指定物体）启动一条自动取放序列，把物体 A、B、C 分别夹起并放到各自的放置点。每条序列由固定的十个阶段组成：移到物体上方 → 垂直下降 → 合爪 → 停顿 → 垂直抬起 → 平移到放置点上方 → 垂直下降 → 松爪 → 停顿 → 垂直撤离。三个物体的初始位置与放置位置各不相同，且每个物体在 x、y 两个方向上都有明显位移（位置表里逐条标了 $\Delta x / \Delta y$ ），保证动作看得出区别而不是原地起落。

实现方式是“点位线性插值 + `getAngleEx()` 反解”：插值是在笛卡尔空间做的，所以下降与抬起是真正的垂直运动；插值点反解成关节角后写进 `Pos.ser`，只刷新一次正解。启动前会像绘图一样把整条路径采样校验一遍（在 `limit` 内、可反解、关节角没被限位吸附、相邻采样点反解分支不跳变），不通过就拒绝启动且不改变 `Pos`；序列本身非阻塞，每轮 `loop()` 推进一步，期间串口与指示灯照常工作，只有动作类命令被让位（表 4）。

★ 本机默认 WEARM_BUTTON_PINS=0：完全不读 D2~D5（连 `pinMode` 都不做），四个功能一律走串口命令。原因是本机没有独立按键，而右摇杆推到底会机械压合它自带的轻触开关，那颗开关曾接在 D2 上，会把“把手推到最前”误判成按键 1（循环取放）。改回 1 即恢复物理按键接线，其它文件一行都不用动。

五、任务四：运动学模型（constant_and_positions.h/.cpp）

模型把肩关节放在原点，上臂与下臂等长 $L1=L2=20$ 个单位，`armheight=0`；基座角 `b` 决定水平面内的朝向，关节角 `r`、`c` 决定手臂平面内的姿态。正解为： $\alpha=r$ 、 $\beta=r-c$ 、 $x_{\text{planar}}=L1\cos(\alpha)+L2\cos(\beta)$ 、 $z=L1\sin(\alpha)+L2\sin(\beta)+\text{armheight}$ ，再按 $\theta=90-b$ 旋转到三维： $x=x_{\text{planar}}\cos(\theta)$ 、 $y=x_{\text{planar}}\sin(\theta)$ 。标定校验用两点核对： $(r,c)=(90,90)$ 时末端 (20.00, 20.00)， $(90,105)$ 时 (19.32, 14.82)，与实机一致。

反解是闭式解且只用 b/r/c: $\text{angle1}=90^\circ-\text{atan2}(y,x)$; $\cos(c)=(R^2-L1^2-L2^2)/(2*L1*L2)$; $\alpha=\text{atan2}(z-\text{armheight}, \rho)+\text{atan2}(L2*\sin(c), L1+L2*\cos(c))$; 夹具角不参与反解。解出后回代正解检查残差, 超过 0.05 个单位的解整组丢弃, 宁可不动作也不给出错误姿态。

可达空间用 limit 做软护栏: $x, y \in [-40, 40]$, $z \in [-20, 40]$ (与实测包络一致, 留 0 余量); isReachable() 供各入口在动作前判断。完整推导 (含坐标系定义、公式来源、误差分析) 另见 docs/kinematics_derivation.docx。

六、任务五：绘图 (draw_control.h/.cpp, 第 1134 - 1419 / 3964 - 5362 行)

铅笔固定在夹具上, 绘图模块的任务是“决定下一步走到哪个坐标”。一共 7 个任务 (表 2), 用串口 F 依次循环, 也可以先 F 选好再 D 开始。切换任务时串口回 OK F=<标签>, 所以上位机 (或评审) 随时能知道当前选的是哪个图形。

表 2 绘图的 7 个任务

编号	任务 (串口 F 循环顺序)	轨迹来源	串口标签
0	直线	内置 2 顶点	LINE
1	字母 N	内置 4 顶点	N
2	三角形	内置 4 顶点 (闭合)	TRI
3	字母 Z	内置 4 顶点	Z
4	字母 V	内置 3 顶点	V
5	五点折线	五个示教点, 按弧长等分	POLY
6	五点曲线	五个示教点, 向心 Catmull-Rom 曲线	CURVE

内置图形 (直线、N、三角形、Z、V) 的顶点以“归一化坐标 $u, v \in [-1, 1]$ ”存放在 PROGMEM 里, 画之前按 中心 + $u*$ 半宽、 $v*$ 半宽 映射到纸面, 所以改图形大小只改一个数 ($n<\text{数字}>$) 而不必重画表; 纸面高度与中心分别用 $p<\text{数字}>$ 、 $o<x>$, $<y>$ 标定。五点折线与五点曲线则来自示教 (见第七节), 分别按“折线”和“曲线”的方式重新采样。

绘制过程是一个显式状态机 (表 3), 每轮 loop() 推进一步: 先回待机位, 抬笔平移到起点上方, 垂直落笔, 沿轨迹走完, 抬笔, 最后回到待机位。每个采样点都交给 moveToPoint() 完成反解、关节速率限制、写 Pos 与正解刷新, 所以绘图与其它入口共用同一份运动基元。

表 3 绘图状态机

阶段	含义	退出条件
IDLE	空闲 (不占用机械臂)	收到 D 或按键请求
HOME	关节空间插值回待机位 ($50^\circ/s$, 0.4° 容差)	三轴到位
TRAVEL	抬笔状态平移到轨迹起点上方	到达
PLUNGE	垂直落笔到纸面高度	到达
PATH	沿轨迹走 (梯形速度 + 关节速率限制)	弧长走完
LIFT	抬笔 DRAW_LIFT_DZ (4.0 单位)	到达
TEACH	五点示教: 摇杆点动笔尖、按键记点	记满 5 点或按键结束
TEACH_WAIT	第 5 点记完后的 700ms 等待 (可撤销)	超时自动结束示教
RETURN	抬笔后回到待机位	到位后回 IDLE

命令: D 开始 (内置图形直接画; 折线/曲线先进入五点示教)、G 记一个示教点、E 撤销一个、Q 暂停、U 继续、W 取消; 物理按键在绘图期间被转交为: 按键 1 = 记录/暂停、按键 2 = 撤销/继续、按键 3 = 取消、按键 4 = 结束示教。暂停在空闲、示教与示教等待阶段没有意义, 返回 BUSY; 继续时会先把速度复位为 0, 避免恢复瞬间猛冲; 取消在绘制中会先抬笔再回待机, 防止铅笔在纸上拖出多余的线。

七、背景机制之一：示教点记录

考试要求“用示教方式给出五个点，再画折线/曲线”，所以示教阶段记录的是笔尖的笛卡尔坐标（Pos.rec 的 x/y/z），而不是关节角——操作者的目标是“把笔尖对到纸面上的某个点”，用关节角没法直观对准。记录流程分四步：

- 1) **进入示教**：D 选中折线/曲线后，固件先回待机位，再进入示教阶段（阶段 TEACH）。
- 2) **点动笔尖**：左手柄左右（A0）改笔尖 x，左手柄前后（A1）改笔尖 y，右手柄前后（A3）改笔尖 z（前推 = 下压）；右手柄左右（A2）默认**不响应**，因为夹具里夹着笔，动它会带歪笔尖（要恢复旧行为把 DRAW_JOG_TOOL_ENABLE 改 1）。
- 3) **每格位移与节奏**：单格步长 = $\text{DRAW_JOG_STEP_MAX}(0.25) \times \text{偏转比例}$ ，下限 0.02 单位；间隔与摇杆同一条规则（固定间隔），所以示教时也吃 H/L 调速档。每个点用“越界夹取”模式写入（MOVE_XYZ_JOG），即使某一步超出关节能力也只是走少一点，不会报错中断。
- 4) **记点与收尾**：按键 1（或串口 G）记一个点，最多 5 个（DRAW_TEACH_MAX_POINTS）；记满 5 个后自动进入 700ms 等待，此时还能撤销（按键 2 / E）；等待结束自动进入“测量 + 校验”，通过才开始绘制。

示教期间允许撤销是因为五个点里只要有一个点没对准，重做整个示教代价太高。若点数不足 2 个，结束示教会被拒绝（REJECTED），机械臂不会乱走。

八、背景机制之二：轨迹生成

轨迹生成的输入是“若干个控制点”，输出是“一串纸面采样点”，中间三步都由纯几何代码完成（path_core.h 与 draw_control.cpp 的测量/校验函数），与硬件无关，所以能在 PC 端用探针逐个数值验证。

- 1) **建点**：内置图形把归一化顶点映射到纸面；示教任务直接用 5 个示教点。
- 2) **测长**：折线用 polyMeasure() 逐段求长；曲线用向心 Catmull-Rom ($\text{knotDelta} = \sqrt{|\mathbf{P}_i - \mathbf{P}_{i+1}|}$)，下限 0.001)，curveSpan()/curvePoint()/curveSpeed() 给出参数 u 处的位置、速度与切线。measurePath() 把总弧长与每个控制点的累计弧长记下来，供按弧长定位使用。
- 3) **按弧长取点**：折线用 polyPointAt(s) 在多段间定位；曲线用 curveSpeed() 把“走多少弧长”换算成“u 前进多少”。这样两种轨迹在速度规划眼里是同一种东西：一个总弧长 + 一个“当前位置”。
- 4) **启动前校验**（validatePath()）：机械臂从待机位出发，依次检查“待机位 → 起点上方”“垂直落笔”“轨迹本体”“终点抬笔”四段：每个采样点都要在 work 空间内、能反解、且反解结果没有被限位吸附；曲线相邻两个采样点的反解分支差不得超过 25°（防止肘部在两点之间突然翻转）。任一条件不满足就**拒绝开始**，此时 Pos 一个字节都不改。
- 5) **抬笔高度**：liftZFor() 从“纸面高度 + 4.0 单位”开始，每次降 0.5 单位试探，直到该高度的点可解、可到达为止，兜底返回纸面高度；这样即使某个图形靠外，抬笔平移也不会撞到关节极限。

采样密度与校验次数都有上限（采样步长 0.25 单位、单条路径最多 96 个采样点），保证一次校验耗时可控、不会让串口响应变卡。

九、背景机制之三：速度控制

速度控制在三个层次上同时生效，任何一层先到限就按那一层走：

- **档位（人机层）**：三个速度档（慢/中/快）各自带一组参数——满偏时的每格关节角 stepSize、以及该档固定的步进间隔 stepDelayMs（慢/中/快 = 80/40/20 ms）。间隔是常量，只有步长按偏转比例缩放，所以推得越狠只是“一步迈得更大”，节奏恒定。串口 H/L 升/降一档，摇杆与示教点动都读它，所以“调速”对所有手动动作同时生效。

- **梯形速度（轨迹层）**：绘制时 `speedLimit()` 取 $v_{End} = \sqrt{2 * \text{加速度} * \text{剩余弧长}}$ 与 $v_{Ramp} = \text{当前速度} + \text{加速度} * dt$ 的较小值，上限 `DRAW_V_MAX=3.0` 单位/秒，加速度 `DRAW_ACCEL=6.0`——起步与收尾自动减速，中间匀速。
- **关节速率（安全层）**：`moveToPoint()` 会检查每一步是否让任一关节超过该模式的角速度上限（绘制 $70^\circ /s$ 、回中 $50^\circ /s$ 、示教点动 $200^\circ /s$ ），超过就整点拒绝或夹取。绘图被拒绝时会把本步弧长折半重试，最多 4 次，仍不行则本轮不动、也不推进弧长，从而“慢下来”而不是跳步。
- **时间步长**：每轮的实际耗时被夹在 $5ms \sim 100ms$ (`applyDtSec()`)，一轮超过 50ms (`DRAW_TICK_MAX_MS`) 按 50ms 计，避免一次卡顿导致下一步走得过远；单步位移另有 0.02 单位的硬上限。

为什么不在“一轮里再拆成多个小步”来求更精确的轨迹：在不做这些额外拆分的版本里，链接期优化 (LTO) 能省下约 662 字节 flash，而 $1 \sim 5ms$ 的周期算出的步长本来就只有 $0.003 \sim 0.015$ 单位，远小于 0.02 的上限，不拆也够平滑。这是 flash 余量只有几百字节时的取舍。

十、背景机制之四：异常处理

固件把所有“可能让机械臂做出意外动作”的路径都提前挡住，原则是**宁可拒绝，也不给出错误姿态**。已实现的保护如下：

- **关节越界**：所有写入都过 `clampServoAngles()`；手动步进遇到行程尽头由 `moveJointStep()` 返回 `MOVE_AT_LIMIT` 原地停住，不顶死舵机。串口角度指令越界只夹取 ($x_{200} \rightarrow 180$)，不报错、不中断当前动作。
- **反解无解或解不唯一/被吸附**：`getAngleEx()` 回代正解算残差，超过 0.05 单位整组丢弃；路径校验中还会逐个采样点检查“反解出来的角是否被硬限位吸附过”以及“相邻采样点的解分支是否跳变 ($>25^\circ$)”。
- **启动前整条路径校验**：绘图与取放都会先把整条轨迹采样验证一遍（在 work 空间内、可反解、无吸附、无分支跳变），任一点不通过就拒绝启动（串口 `REJECTED`），并且**不改动 Pos**，机械臂保持原姿态。
- **忙让位**：表 4 是完整的让位矩阵。取放/录制/播放/绘图各自有独占资源，冲突方拿到 `BUSY`，唯一始终有效的是 H/L 调速，方便演示时随时改变节奏。
- **串口输入保护**：行缓冲 40 字节，超长行整行丢弃（回 `DISCARD`）；一行超过 300ms 没收到换行则丢弃该半行，不会把两次输入粘在一起执行；语法错误回 `ERR`；命令属于未编译/已关闭的模块时回 `REJECTED/OFF`。串口字节**只有一个读者** (`serialProtocolLoop()`)，避免两个函数各吃一半数据造成“命令无响应”。
- **卡死保护**：绘制过程中若连续 500 次推进 (`DRAW_STALL_LIMIT`) 位置几乎没变 ($<1e-6$ 单位)，判定为机械卡死，主动结束任务并回 `REJECTED`，避免电机长时间堵转。
- **录制数据不合格**：结束录制时若没有走出足够位移，数据丢弃并回 `DISCARD`，播放时若没有录制数据回 `EMPTY`，两者都会用短信息说明原因，不会出现“按下没反应”。

表 4 忙让位矩阵（√ 有效，× 被拒并回 `BUSY`/让位）

正在执行	摇杆	串口动作指令	按键	串口 H/L 调速
取放序列 A/B/C	×	×	×	√
录制（按键 2/R）	√	×	×	√
播放/回中（按键 3, 4/P, M）	×	×	×	√
绘图（D 之后的所有阶段）	×（示教阶段除外）	×	×（绘图键本身有效）	√

另外，为了让“没反应”这种情况不可能出现，所有已接受的命令都会回一行短回复：`OK`（含 `OK #` 夹爪角度、`OK F=<模式>` 图形标签）、`BUSY`、`REJECTED`、`OFF`、`DISCARD`、`EMPTY`、`ERR`。回复层刻意不

用 `Serial.print()` 打印数字——那会把 Arduino 约 9.5KB 的数字/浮点格式化代码拉进镜像，而是直接写串口数据寄存器。

十一、验证情况与已知限制

本版固件在 PC 端做过两轮验证，均在修改后重新执行：**9 个编译单元在 `-Wall -Wextra -Wshadow -Wconversion` 下零警告；11 个回归探针全部通过**，覆盖舵机脉冲时序、运动学正反解、关节步进、摇杆映射、按键录制、取放路径、绘图 7 任务的几何与串口协议（含忙守卫、越界夹取、分片输入与超长行）。AVR 实编容量（`arduino-cli` 不可用，用 `avr-gcc 7.3.0 + Arduino core 1.8.8` 复核）：**三功能全开时 32114 字节 flash / 1457 字节 SRAM**，占 Uno 可用 flash 的 98.0%，余量 142 字节；关闭按键为 28922 字节、关闭绘图为 19280 字节。

已知限制：1) 以上验证全部在 PC 端与编译器层面完成，**未在实体 Uno 上烧录验证**；2) 三个功能同时开启时 flash 余量只有 142 字节（摇杆手感修正又用掉 202 字节），继续加功能必须先在 `weArm_config.h` 关掉一个模块；3) 内置图形的尺寸与中心需要在实机上用 n/o/p 标定一次，因为纸面高度取决于夹具与笔的实际装配；4) 示教点动默认不驱动夹具（A2 不响应），若需要在示教时开合爪子，把 `DRAW_JOG_TOOL_ENABLE` 改为 1。